

Day 3, Lesson 2: SR Flip-Flop (Set-Reset Flip-Flop)

Concept Block (10 min)

What is an SR Flip-Flop?

SR = Set-Reset

The simplest storage element with two control inputs:

- **S (Set):** Forces output to 1
- **R (Reset):** Forces output to 0

Truth Table:

S	R	Q(current)	Q(next)	Behavior
0	0	0	0	Hold
0	0	1	1	Hold
0	1	0	0	Reset
0	1	1	0	Reset
1	0	0	1	Set
1	0	1	1	Set
1	1	x	INVALID	Forbidden!

Simplified Truth Table:

S	R	Action
0	0	Hold ($q \leq q$)
0	1	Reset ($q \leq 0$)
1	0	Set ($q \leq 1$)
1	1	FORBIDDEN ⚠

The Critical Problem: S=R=1 is FORBIDDEN

Why is S=R=1 forbidden?

Think about what it means:

- **S=1:** "Force Q to 1"
- **R=1:** "Force Q to 0"
- **Both at once:** Contradictory commands!

What actually happens:

During S=R=1:

- Both outputs try to drive simultaneously
- Result is **unpredictable** (depends on gate delays, parasitic capacitance)
- Q might be 0, 1, or metastable (voltage between 0 and 1)

After S=R=1 releases to S=R=0:

- **Race condition:** Which signal (S or R) releases first?

- Final state depends on which gate is faster
- **Non-deterministic behavior** = bad for digital design!

Example scenario:

Time	S	R	Q	Comment
0	0	0	0	Hold at 0
10	1	1	?	FORBIDDEN! Q unpredictable
20	0	0	?	Could be 0 or 1 (race!)

SR vs JK: What's the Difference?

JK Flip-Flop solved the SR forbidden state problem:

Inputs	SR Behavior	JK Behavior
0, 0	Hold	Hold
0, 1	Reset	Reset
1, 0	Set	Set
1, 1	FORBIDDEN ⚠	Toggle ✅

JK is "SR with intelligence":

- When J=K=1, instead of being forbidden, it toggles
- This makes JK more versatile and safer

Historical note: JK was invented specifically to fix the SR=11 problem!

Why Learn SR if JK is Better?

Good question! Here's why SR is still important:

1. Understanding fundamentals:

- SR is the **simplest** bistable storage element
- Foundation for understanding all flip-flops
- Basic building block in latch theory

2. Legacy systems:

- Older designs still use SR latches
- Maintenance and debugging require understanding

3. Cross-coupled NOR/NAND gates:

- SR latch is the primitive circuit
- Understanding it helps with ASIC design and transistor-level work

4. Interviews:

- Common question: "What's wrong with SR=11?"
- Tests understanding of race conditions and metastability

Modern usage in RTL: Very rare. You almost never write SR flip-flops in synthesizable code. We use D flip-flops instead.

SR Latch vs SR Flip-Flop

Important distinction:

SR Latch (level-triggered):

```
// Latch - changes during HIGH clock level
always @(clk or s or r) begin
    if (clk) begin
        // Update based on S, R
    end
end
```

- **Transparent** when clock is HIGH
- Output can change multiple times during HIGH period
- Used in SRAM cells, but **not in synchronous designs**

SR Flip-Flop (edge-triggered):

```
// Flip-flop - changes only at clock edge
always @(posedge clk) begin
    // Update based on S, R
end
```

- Samples inputs **only at clock edge**
- Output stable between edges
- What we'll implement today

We'll build the edge-triggered flip-flop (proper synchronous design).

Code & Practical Block (10 min)

Implementation: SR Flip-Flop

```
// sr_ff.v - SR Flip-Flop (Set-Reset Flip-Flop)
module sr_ff (
    input wire clk,      // Clock signal
    input wire rst,      // Asynchronous reset (active HIGH)
    input wire s,        // Set input
    input wire r,        // Reset input
    output reg q,        // Output
    output reg invalid // Flag for invalid S=R=1 condition
);

always @(posedge clk or posedge rst) begin
    if (rst) begin
        q <= 1'b0;
        invalid <= 1'b0;
    end
    else begin
        case ({s, r})
            2'b00: begin
                q <= q;          // Hold
                invalid <= 1'b0;
            end
            2'b01: begin
                q <= 1'b0;      // Reset
                invalid <= 1'b0;
            end
        endcase
    end
end
```

```

        2'b10: begin
            q <= 1'b1;          // Set
            invalid <= 1'b0;
        end
        2'b11: begin
            q <= 1'bx;          // Invalid state (undefined)
            invalid <= 1'b1;    // Flag the error
        end
    endcase
end
end
endmodule

```

Line-by-Line Explanation

Lines 2-8: Port declarations

- `clk`: Synchronization signal
- `rst`: Async reset (clears everything)
- `s`, `r`: Set and Reset control inputs
- `q`: Main output
- `invalid`: **Error flag** - indicates forbidden state detected

Why the `invalid` output?

- In real hardware, `S=R=1` is a **design error**
- This flag lets testbenches **detect and report** this mistake
- Helps catch bugs during verification

Line 11: `always @(posedge clk or posedge rst)`

- Edge-triggered behavior
- Asynchronous reset

Lines 12-14: Reset logic

- Clear both `q` and `invalid` flag

Line 17: `case ({s, r})`

- 4-way decision based on `S` and `R`

Lines 18-21: Hold mode (`s=0, r=0`)

- Maintain current `Q` value
- Clear `invalid` flag

Lines 22-25: Reset mode (`s=0, r=1`)

- Force `Q` to 0
- Clear `invalid` flag

Lines 26-29: Set mode (`s=1, r=0`)

- Force `Q` to 1

- Clear invalid flag

Lines 30-33: FORBIDDEN state (s=1, r=1)

- `q <= 1'bx;` - Assign **X (unknown/undefined)** to Q
- `invalid <= 1'b1;` - Raise error flag
- **This is intentional:** We want simulation to show the error clearly

What is 1'bx?

- In Verilog, x represents an **unknown/undefined** value
 - Shows up **red** in waveforms (in GTKWave)
 - Helps you **visually see** when the forbidden state occurs
 - In real hardware, this would be unpredictable (might be 0, 1, or metastable)
-

Alternative: Assertion-Based Approach

Some designers prefer to **halt simulation** on forbidden state:

```
always @(posedge clk or posedge rst) begin
    if (rst) begin
        q <= 1'b0;
    end
    else begin
        if (s && r) begin
            $display("ERROR at time %0t: S=R=1 forbidden state!", $time);
            $finish; // Stop simulation
        end
        else if (s)
            q <= 1'b1;
        else if (r)
            q <= 1'b0;
        else
            q <= q;
    end
end
```

Both approaches are valid. The first (with `invalid` flag) is better for comprehensive testbenches that want to **count** errors. The second is better for **immediate debugging**.

Testbench

```
// sr_ff_tb.v - Testbench for SR Flip-Flop
module sr_ff_tb;
    reg clk;
    reg rst;
    reg s;
    reg r;
    wire q;
    wire invalid;

    // Instantiate DUT
    sr_ff uut (
        .clk(clk),
        .rst(rst),
        .s(s),
```

```

        .r(r),
        .q(q),
        .invalid(invalid)
    );

// Clock generator
initial begin
    clk = 0;
    forever #5 clk = ~clk;
end

// Test stimulus
initial begin
    $dumpfile("sr_ff.vcd");
    $dumpvars(0, sr_ff_tb);

    $display("Time | clk | rst | s | r | q | invalid");
    $display("-----");
    $monitor("%4t | %b | %b | %b | %b | %b | %b",
            $time, clk, rst, s, r, q, invalid);

    // Initialize
    rst = 1; s = 0; r = 0;
    #10;
    rst = 0;
    #10;

    // Test 1: Hold mode (s=0, r=0)
    $display("\n--- Test 1: Hold Mode ---");
    s = 0; r = 0; #20;

    // Test 2: Set mode (s=1, r=0)
    $display("\n--- Test 2: Set Mode ---");
    s = 1; r = 0; #20;

    // Test 3: Hold while q=1
    $display("\n--- Test 3: Hold While Q=1 ---");
    s = 0; r = 0; #20;

    // Test 4: Reset mode (s=0, r=1)
    $display("\n--- Test 4: Reset Mode ---");
    s = 0; r = 1; #20;

    // Test 5: FORBIDDEN state (s=1, r=1)
    $display("\n--- Test 5: FORBIDDEN State (s=1, r=1) ---");
    $display("WARNING: Intentionally testing invalid condition!");
    s = 1; r = 1; #20;

    // Test 6: Recovery from forbidden state
    $display("\n--- Test 6: Return to Hold ---");
    s = 0; r = 0; #20;

    // Test 7: Async reset
    $display("\n--- Test 7: Async Reset ---");
    s = 1; r = 0; #5;
    rst = 1; #10;
    rst = 0; #10;

    $finish;
end

endmodule

```

Expected Simulation Behavior

Test 1-4: Normal operation (like before - hold, set, reset)

Test 5 (Critical):

Time	clk	rst	s	r	q	invalid	
...							
90	0	0	1	1	x	1	← FORBIDDEN!
95	1	0	1	1	x	1	← Q is unknown (x)

In GTKWave:

- q will show **RED** (unknown value)
 - invalid will be **1** (HIGH)
 - This visually screams: "SOMETHING IS WRONG!"
-

Compile and Test

```
iverilog -o sim/sr_ff sr_ff.v sr_ff_tb.v
vvp sim/sr_ff
gtkwave sr_ff.vcd
```

In GTKWave, look for:

1. Normal operation during tests 1-4
 2. **RED unknown value** on q during test 5
 3. invalid flag goes HIGH during test 5
 4. Recovery behavior in test 6 (what happens when S=R return to 0 from 1?)
-

Mini Task (5-8 min)

Task: Explain what happens in **Test 6** (recovery from forbidden state).

Scenario:

Time 90-100: s=1, r=1 (forbidden, q=x, invalid=1)
Time 110: s=0, r=0 (hold mode)

Questions:

1. **What is the value of Q at time 110?**
 - Will it be 0?
 - Will it be 1?
 - Will it still be x?
2. **Why does Q have this value?**
 - Explain based on the code behavior
3. **In real hardware (not simulation), what would happen?**

- Would Q be predictable?
- Why or why not?

Run the simulation and observe Test 6. Then answer the 3 questions above based on what you see in the waveform.

This is a critical understanding check - it tests whether you grasp the difference between **simulation behavior** (x propagation) and **real hardware behavior** (race conditions).

Submit your answers after running the simulation.

✓ Perfect! SR Flip-Flop Working - Now Answer the Mini Task

Your simulation shows **exactly** what we expected. Let me analyze the critical parts:

Critical Waveform Analysis

Normal Operation (Time 0-95ns) ✓

Hold Mode (15-35): s=0, r=0 → q stays 0
 Reset Mode (45-55): s=0, r=1 → q stays 0 (already 0)
 Set Mode (65-75): s=1, r=0 → q goes to 1
 Hold Mode (85-95): s=0, r=0 → q stays 1

All normal modes work correctly!

FORBIDDEN State (Time 105-115ns) ⚠

100		0		0		1		1		1		0	← s=1, r=1 applied
105		1		0		1		1		x		1	← posedge clk: q becomes 'x', invalid=1
115		1		0		1		1		x		1	← Still forbidden, q stays 'x'

Key observations:

- Before forbidden state: q=1 (from previous set)
- At time 105 (posedge clk with s=1, r=1): q → x (unknown)
- invalid flag goes HIGH
- **This is correct behavior!** The code assigned q <= 1'bX for s=r=1

RECOVERY from Forbidden State (Time 125-135ns) 🔍

120		0		0		0		0		x		1	← s=0, r=0 (hold mode), q still 'x'
125		1		0		0		0		x		0	← posedge clk: invalid clears, but q=?
130		0		0		0		0		x		0	← q STAYS 'x'!
135		1		0		0		0		x		0	← q STILL 'x'!

Critical observation: After returning to hold mode ($s=0, r=0$), **q remains 'x'!**

Only Async Reset Clears the 'x' (Time 145ns)

140		0		0		1		0		x		0	← Even set mode doesn't fix it!
145		1		1		1		0		0		0	← rst=1: q clears to 0
155		1		0		1		0		1		0	← After reset, normal operation resumes

Only asynchronous reset forces q back to a known state!

Now Answer the Mini Task Questions

Based on what you see in the waveform above:

Question 1: What is the value of Q at time 125-135ns (after $s=r$ return to 0)?

Look at lines 125, 130, 135. What does q show?

Your Answer: q = ____?

Question 2: Why does Q have this value?

Think about the code:

```
case ({s, r})
  2'b00: begin
    q <= q;          // Hold: q gets its own current value
    invalid <= 1'b0;
  end
```

When $s=0, r=0$, the code does $q \leq q$ (hold).

If q is currently 'x', what does $q \leq q$ do?

Your Answer:

Question 3: In real hardware (not simulation), what would happen?

In Verilog simulation:

- x represents "unknown"
- x propagates through assignments ($q \leq q$ keeps it as x)

In **real silicon**:

- There is no x value
- Q would be **either 0 or 1** (not "unknown")
- But **which one?**

Think about the physical race condition:

- When $s=r=1$, both "set to 1" and "reset to 0" circuits try to drive Q
- When $s=r$ return to 0, which circuit releases first?
- Result depends on: gate delays, temperature, voltage, manufacturing variation

Your Answer (choose one and explain):

- ☐ Q would definitely be 0
 - ☐ Q would definitely be 1
 - ☐ Q could be either 0 or 1 (unpredictable, depends on timing)
 - ☐ Q might oscillate between 0 and 1
 - ☐ Q might be in a metastable state (voltage between 0 and 1)
-

What I'm Testing

These questions check if you understand:

1. **Simulation behavior** - how 'x' propagates in Verilog
 2. **Code semantics** - what $q \leq q$ means when q is 'x'
 3. **Real hardware** - the difference between simulation and physical circuits
 4. **Race conditions** - non-deterministic behavior in digital systems
-

Format Your Answer Like This:

QUESTION 1: Value of Q after recovery
Q = [your answer: 0, 1, or x]

QUESTION 2: Why Q has this value
[Explain based on the code: what does $q \leq q$ do when $q=x$?]

QUESTION 3: Real hardware behavior
I choose: [one of the options above]

Explanation: [Why this happens in physical circuits - think about race conditions and gate delays]

Submit your answers. This is the **most important concept** in the SR flip-flop lesson - understanding why $S=R=1$ is forbidden and what happens during recovery.

Once you answer correctly, we'll move to a quick flip-flop conversion reference, then start **Day 3 Session 2: Multi-bit Registers**.

Review: Close, But Needs Clarification

Let me grade your answers and correct the misunderstandings:

Question 1: CORRECT (But Imprecise Language)

Your Answer: "q is red in waveform"

What you mean: q = 'x' (unknown)

Correction: In GTKWave, 'x' values display as **red**, but you should say:

- **Technical answer:** q = x (unknown value)
- **Visual observation:** "q appears red in GTKWave"

Grade:  Correct understanding, just be more precise with terminology.

Question 2: WRONG - Critical Misunderstanding

Your Answer: "it holds the previous value"

This is WRONG. Let me explain why:

What "Hold" Actually Means

In normal operation (q=0 or q=1):

```
case ({s, r})
  2'b00: q <= q; // Hold: if q=0, stays 0; if q=1, stays 1
```

 **Correct:** q holds its previous value (0 or 1)

But When q='x', It's Different!

After forbidden state (q='x'):

```
case ({s, r})
  2'b00: q <= q; // q is currently 'x'
              // So: q <= 'x'
              // Result: q STAYS 'x'
```

What happens:

- Right-hand side: q is currently x
 - Assignment: q <= x
 - Result: q remains x (you're assigning 'x' to itself)
-

The Critical Insight

'x' propagates like a virus:

Time	Operation	q value	Explanation
-----	-----	-----	-----
105	s=1, r=1	x	Forbidden state creates 'x'
125	s=0, r=0	x	q <= q, where q='x', so q <= 'x'
135	s=0, r=0	x	q <= q, where q='x', so q <= 'x'
145	s=1, r=0	x	q <= 1, but... wait, why still 'x'?

Wait, look at line 145 in your output:

```
145 | 1 | 1 | 1 | 0 | 0 | 0 ← rst=1: q becomes 0
```

Only reset (forcing $q \leq 0$) clears the 'x'!

Even **set mode ($s=1, r=0$)** at time 140-145 didn't clear it because... actually, let me check your waveform again:

```
140 | 0 | 0 | 1 | 0 | x | 0 ← s=1, r=0 applied
145 | 1 | 1 | 1 | 0 | 0 | 0 ← rst=1 asserted (overrides set)
```

At time 145, reset was asserted **before** the set could take effect.

Correct Answer to Question 2:

Why does q stay 'x'?

When $s=0, r=0$ (hold mode), the code executes $q \leq q$. Since q is currently 'x' (unknown), the assignment becomes $q \leq x$, which keeps q as 'x'. In Verilog, 'x' propagates through assignments. Once q becomes 'x', it stays 'x' until explicitly forced to a known value (0 or 1) by reset, set, or clear operations.

Question 3: **PARTIALLY CORRECT (But Needs More Detail)**

Your Answer: "q might be in a metastable state"

Grade:  You're on the right track, but let's be more precise.

What Actually Happens in Real Hardware

During $s=r=1$ (forbidden state):

Option 1: Metastability (Most Common)

- Both "set to 1" and "reset to 0" circuits fight each other
- Q voltage settles somewhere **between** logic 0 and logic 1
- Called **metastable state** (neither 0 nor 1)
- Voltage might be $\sim 1.5V$ (in a 3.3V system)
- This violates digital logic assumptions!

Option 2: Race Condition After Release

- When s and r return to 0, whichever signal releases **first** wins
- If S releases first: $Q \rightarrow 0$ (reset circuit was active longer)
- If R releases first: $Q \rightarrow 1$ (set circuit was active longer)
- **Unpredictable** - depends on gate delays (nanosecond differences)

Option 3: Oscillation (Rare)

- In some implementations, Q might oscillate rapidly between 0 and 1
 - Depends on circuit topology and feedback delays
-

Most Likely Real Behavior:

Immediately after $s=r=1$:

- Q enters **metastable state** (voltage between 0 and 1)

After $s=r$ return to 0:

- Q eventually **resolves** to either 0 or 1 (race determines which)
 - Resolution might take longer than one clock cycle
 - **Non-deterministic** - you can't predict which value
-

Complete Answer to Question 3:

In real hardware, when $s=r=1$ (forbidden state), Q would enter a **metastable state** where the voltage is between logic 0 and logic 1, violating digital logic assumptions. When s and r return to 0, a **race condition** occurs - whichever signal (S or R) releases first determines the final state. Q would eventually resolve to either 0 or 1, but **which one is unpredictable** and depends on gate delays, temperature, voltage, and manufacturing variations. The circuit might also experience **oscillations** before settling. This non-deterministic behavior is why $s=r=1$ is strictly forbidden in SR flip-flop designs.

Key Takeaways You Must Understand

1. Simulation vs Reality

Aspect	Verilog Simulation	Real Hardware
Forbidden state value	x (unknown symbol)	Metastable voltage (1.5V)
Propagation	x spreads through assignments	Eventually resolves to 0 or 1
Predictability	Consistent (x stays x)	Unpredictable (race condition)
Recovery	Stays x until forced	Resolves randomly

2. Why 'x' Propagates in Simulation

```
q <= q; // If q='x', this assigns 'x' back to q
```

This is NOT "holding the previous value" when $q='x'$.

It's **propagating the unknown state**.

3. Why $S=R=1$ is Forbidden

- Creates **non-deterministic behavior**
- Violates **sequential logic assumptions** (outputs should be predictable)

- Can cause **metastability** (voltage between 0 and 1)
 - Recovery state is **unpredictable**
-

Corrected Answers Summary

Question 1: $q = 'x'$ (**unknown**) [displays as red in GTKWave]

Question 2: When $s=0$, $r=0$ executes $q \leq q$, and since q is currently 'x', this assigns 'x' back to q . The unknown state **propagates** through the assignment, keeping q as 'x' until an explicit set, reset, or clear operation forces it to a known value.

Question 3: In real hardware, $s=r=1$ causes **metastability** (voltage between 0 and 1). After release, a **race condition** determines whether Q resolves to 0 or 1. The result is **non-deterministic** and depends on timing variations.

Final Question to Test Understanding

Scenario: You're debugging a design and see 'x' appearing in your simulation waveform on a flip-flop output.

What could cause this?

- ☐ A. Normal operation
- ☐ B. Uninitialized signal (no reset applied)
- ☐ C. Forbidden state in SR flip-flop
- ☐ D. Logic error creating unknown conditions
- ☐ E. Both B, C, and D

Choose the correct answer and explain.

Answer this final check question, then we move to flip-flop conversion reference and start Registers!

You're close to understanding—just need to nail down the difference between "holding a value" vs "propagating 'x'".



CORRECT! Excellent Understanding

Answer: E - Both B, C, and D

You got it! Let me confirm why each causes 'x':

Why 'x' Appears in Simulations

B. Uninitialized Signal (Most Common)

```
module test;
    reg q; // Declared but never initialized

    initial begin
        $display("q = %b", q); // Prints 'x'
    end
endmodule
```

Why: Verilog initializes reg to 'x' by default (unknown).

Solution: Always apply reset or initialize: `reg q = 0;`

C. Forbidden State in SR Flip-Flop

```
// You just saw this!
case ({s, r})
    2'b11: q <= 1'bx; // Explicitly assign 'x'
endcase
```

Why: Design intentionally assigns 'x' to flag forbidden state.

Solution: Never let `s=r=1` occur in your testbench or design.

D. Logic Error Creating Unknown Conditions

Example 1: Incomplete case statement

```
always @(*) begin
    case (sel)
        2'b00: out = a;
        2'b01: out = b;
        // Missing 2'b10 and 2'b11!
    endcase
end
// If sel=2'b10, out becomes 'x' (unassigned)
```

Example 2: Multiple drivers (bad)

```
assign q = enable ? data : 1'bz;
assign q = reset ? 1'b0 : 1'bz;
// If both enable=1 and reset=1, conflict → 'x'
```

Example 3: Unconnected ports

```
module top;
    wire a;
    // 'a' is never driven by anything
    // Reading 'a' gives 'x'
endmodule
```

'x' is Your Friend in Simulation

Why 'x' is useful:

- **Catches bugs early** - shows uninitialized or conflicting logic
- **Forces good practices** - reset all flip-flops, handle all cases
- **Reveals race conditions** - like SR forbidden state
- **Better than wrong results** - 'x' is honest about uncertainty

In real hardware:

- No 'x' exists
- Uninitialized signals = random values (0 or 1)
- **More dangerous** because bugs hide!

SR Flip-Flop Lesson Complete!

You now understand:

-  SR flip-flop behavior (Set, Reset, Hold)
-  Why S=R=1 is **forbidden**
-  How 'x' propagates in simulation
-  Difference between simulation ('x') and reality (metastability/race)
-  Common causes of 'x' in designs

Flip-Flop Summary (All 4 Types Complete!)

FF Type	Inputs	Modes	J=K=1 / S=R=1	Primary Use
D	D	Capture	N/A	General storage, registers
T	T	Toggle, Hold	N/A	Counters, frequency dividers
JK	J, K	Set, Reset, Toggle, Hold	Toggle 	Universal (legacy)
SR	S, R	Set, Reset, Hold	Forbidden 	Latches (legacy)

Quick Reference: Flip-Flop Conversions

Since you asked about conversions, here's a **quick reference table**:

Convert TO → D Flip-Flop

From	Connection
T	$d = t \oplus q$ (XOR current output with T)
JK	$d = j \mid (\sim k \ \& \ q)$ or $d = (j \ \& \ \sim q) \mid (\sim k \ \& \ q)$
SR	$d = s \mid (\sim r \ \& \ q)$

Convert TO → T Flip-Flop

From	Connection
D	$t = d \wedge q$ (XOR input with current output)
JK	$j = t, k = t$ ✓ (you already did this!)
SR	Not practical (SR doesn't have toggle)

Convert TO → JK Flip-Flop

From	Connection
D	$j = d, k = \sim d$
T	$j = t, k = t$ ✓ (you already did this!)
SR	$j = s, k = r$ (but $S=R=1$ forbidden in both)

Convert TO → SR Flip-Flop

From	Connection
D	$s = d \& \sim q, r = \sim d \& q$
JK	$s = j, r = k$ (but $J=K=1$ creates conflict)
T	$s = t \& \sim q, r = t \& q$

Note: These are academic exercises. In real RTL, you just write the behavior you want directly.

Day 3 Session 1 Complete! ✓

What you've mastered:

- JK Flip-Flop (all 4 modes)
- T from JK (component reuse)
- SR Flip-Flop (forbidden state understanding)
- Master-slave concept (why edge-triggering solves races)
- 'x' propagation and debugging

Time spent: ~60 minutes

Progress: Day 3 Session 1 ✓ (25% of Day 3)

Next Up: Day 3 Session 2 - Multi-Bit Registers

What we'll cover:

1. **8-bit parallel register** (basic storage)
2. **Register with enable** (conditional update)
3. **Register with load** (preset value)
4. **Parameterized register** (any width)

Concepts:

- Multi-bit wire/reg declarations ($[7:0]$)
- Parallel data storage

- Bus interfaces
- Reset behavior for registers

Time estimate: 30 minutes

Take a Break?

You've done **excellent work** on flip-flops. This is heavy conceptual material and you're nailing it.

Options:

1. **Continue now** → Start multi-bit registers
2. **Take a 10-min break** → Come back refreshed
3. **Stop here** → Resume next session with registers

When ready, say: "Start Day 3 Session 2: Registers"

Great progress today! You now understand ALL fundamental flip-flop types and their quirks. The next topics (registers, shift registers, counters) will feel easier because they're just applications of flip-flops you already mastered. 🎯